

RE2: End-to-End Machine Learning Project – Guided Hands-On Review & Reflection

Instructor:

Dr. Diala Ezzeddine

Team Members:

1. Phu Han
2. Baran Onalan

1. Overview of the example Project:

Problem Addressed:

The project aims to build a model to predict housing price using the California Housing dataset, the dataset includes metrics such as population, median income, and median housing price for various districts...

Type of Machine Learning Problem:

This is a supervised learning task because the model is trained with labeled data (“median house value”). More specifically, it is a regression problem because the goal is to predict a continuous numerical value.

Overall Goal:

The overall goal of the model is to learn a function that generalizes well to unseen districts, allowing it to make accurate price predictions that could support business or policy decisions (for example: real estate valuation or investment planning).

2. Reproducing the End-to-End Workflow:

Problem Framing:

The goal is to predicting house values and it is a supervised regression problem with a performance metric RMSE.

Data Loading and Exploration:

The data is fetched from a remote repository. Exploration includes using `housing.info()`, `housing.describe()`,... counting categories value and plotting histograms to understand feature distributions and identify outliers, visualizing correlations, and identifying potential data issues.

Data Pre-processing:

Handling missing values, encoding categorical features (ocean proximity), feature scaling, and building preprocessing pipelines. I also notice feature distributions, including checking whether variables are normally distributed and applying transformations when necessary.

Model Training:

Several models are explored, including Linear Regression, and tree-based models like `DecisionTreeRegressor`, and `RandomForestRegressor`.

Evaluation:

Models are evaluated using Root Mean Squared Error (RMSE) to measure prediction accuracy. Techniques like cross-validation and GridSearchCV are used to fine-tune the hyperparameters of the models.

Which steps were straightforward, and which required more effort to understand?

Steps that were straightforward are data loading and exploration, model training while data preprocessing, pipelines and column transformers (this was my first time trying to understand pipelines and column transformer) and evaluation required more effort to understand.

3. Learning from the Code:

One of the most impactful practices in the code is the use of Pipeline and ColumnTransformer. These allow for a clean, reproducible sequence of transformation steps that can be applied to both training and test data without manual repetition.

Running the code made several concepts much clearer compared to only reading the book. In particular, the value of pipelines became more concrete once I saw how they prevent data leakage and simplify experimentation.

Some code sections initially felt confusing, such as the interaction between Pipeline, ColumnTransformer, and custom transformers. However, after modifying the code and rerunning experiments, the abstraction started to make sense to modularize complex workflows.

The book also demonstrates the importance of using StratifiedShuffleSplit. More specifically, Stratified Sampling ensures that the training and test sets are representative of the overall population based on a critical feature (like median income).

Executing the code also helped me gain practical details that are easy to miss when reading, such as how small preprocessing changes can significantly affect model performance and how cross-validation provides a more reliable evaluation than a single train-test split also how they had an attention on the data that was capped.

4. Challenges and Debugging:

One challenge was the difference between the textbook and the notebook from [GitHub repository](#). Some code examples mentioned in the book were missing or commented out in the notebook, and certain sections could not be run directly due to bugs or version differences. As a result, I had to spend additional time identifying what was wrong and fixing the code before I could experiment with it. This experience showed that even well-documented ML examples often require debugging and adaptation to the actual working environment.

5. Team Experience:

Our team worked on the project individually, each member running the notebook and trying to understand the workflow on their own. I think this allowed everyone to gain hands-on experience with the full pipeline rather than focusing on only a single part of the project. Whenever anyone faced an issue with the notebook, such as errors during preprocessing, confusion about pipelines, or unexpected model behavior, we communicated with each other to ask questions, debug together, and explain concepts. This collaborative problem-solving approach helped resolve issues more efficiently and ensured that all team members reached a similar level of understanding.

6. Reflection & Preparation for the Quarter-Long Project:

What are the top 2–3 takeaways from this guided end-to-end project?

The top takeaways from this project are the importance of a structured end-to-end ML workflow, how preprocessing and evaluation choices impact model performance, and why reproducibility through clean pipelines matters in real-world projects.

How does this experience prepare you for the upcoming quarter-long ML team project?

This experience prepares me well for a quarter-long ML team project by building confidence in problem framing, preprocessing, and evaluation. I now feel more comfortable navigating scikit-learn pipelines and debugging model issues. However, feature engineering and model selection still feel challenging and are areas I want to improve further.

What parts of the ML workflow do you feel more confident about now? What still feels challenging?

I now feel more confident about what a hands-on machine learning project looks like in practice, especially the overall structure of an end-to-end workflow. I also learned the importance of taking clear notes during development, since it is easy to forget the reasoning behind certain design or preprocessing decisions later. What still feels challenging is deciding which features or preprocessing choices will lead to the most meaningful performance improvements.

My Notebook on GitHub: https://github.com/htnpnu/CPSC-5310-01-Machine_Learning/blob/main/w02/02_end_to_end_machine_learning_project.ipynb